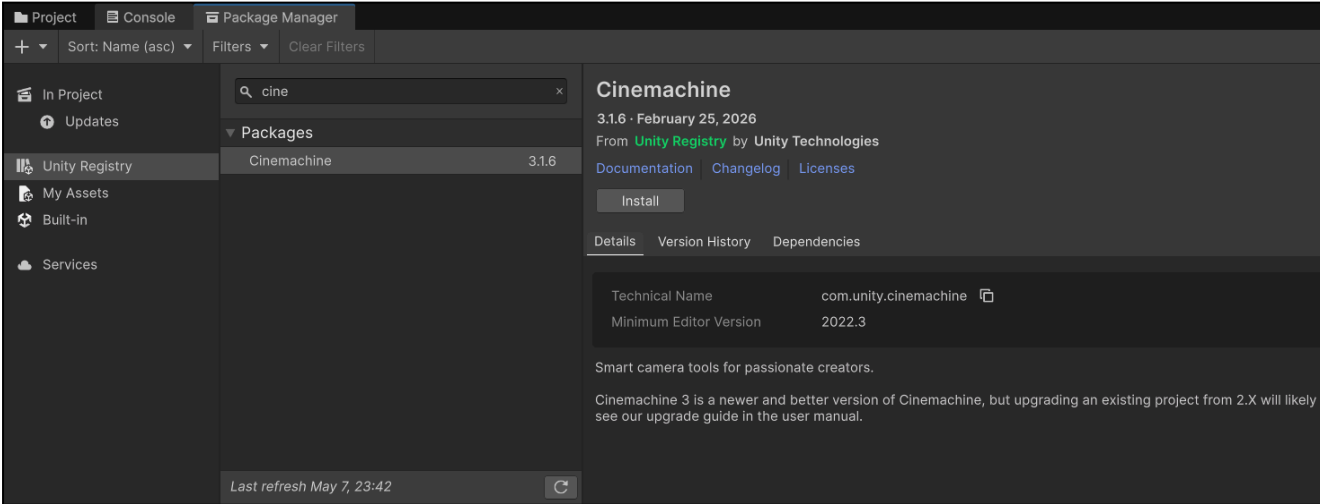


|                                                                                    |                                                                                                                                                                                        |                                                                                     |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   | <p align="center"><b>UNIVERSIDAD NACIONAL DE SAN AGUSTIN</b><br/> <b>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS</b><br/> <b>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</b></p> |  |
| <b>Formato:</b> Guía de Práctica de Laboratorio / Talleres / Centros de Simulación |                                                                                                                                                                                        |                                                                                     |
| <b>Aprobación:</b> 2022/03/01                                                      | <b>Código:</b> GUIA-PRLE-001                                                                                                                                                           | <b>Página:</b> 1                                                                    |

## INFORME DE LABORATORIO 04

| INFORMACIÓN BÁSICA                                         |                                                        |                             |      |                       |    |
|------------------------------------------------------------|--------------------------------------------------------|-----------------------------|------|-----------------------|----|
| <b>ASIGNATURA:</b>                                         | Computación Gráfica, Visión Computacional y Multimedia |                             |      |                       |    |
| <b>TÍTULO DE LA PRÁCTICA:</b>                              |                                                        |                             |      |                       |    |
| <b>NÚMERO DE PRÁCTICA:</b>                                 | 04                                                     | <b>AÑO LECTIVO:</b>         | 2026 | <b>NRO. SEMESTRE:</b> | IX |
| <b>FECHA DE PRESENTACIÓN</b>                               | 10/05/26                                               | <b>HORA DE PRESENTACIÓN</b> |      |                       |    |
| <b>ESTUDIANTE:</b><br>Ponce de Leon Aguilar, Marco Antonio |                                                        |                             |      | <b>NOTA:</b>          |    |
| <b>DOCENTE:</b><br><i>Rene Alonso Nieto Valencia</i>       |                                                        |                             |      |                       |    |

| SOLUCIÓN Y RESULTADOS                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS</b></p> <p><b>1. Instalación de CineMachine</b></p> <p>Primero instalamos el paquete de <b>CineMachine</b> desde el <b>Package Manager</b> para poder utilizar diversas funciones de cámara en el proyecto.</p>  |

|                                                                                   |                                                                                                                                                                                                     |                                                                                     |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  | <p style="text-align: center;"><b>UNIVERSIDAD NACIONAL DE SAN AGUSTIN</b><br/> <b>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS</b><br/> <b>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</b></p> |  |
| Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación       |                                                                                                                                                                                                     |                                                                                     |
| Aprobación: 2022/03/01                                                            | Código: GUIA-PRLE-001                                                                                                                                                                               | Página: 2                                                                           |

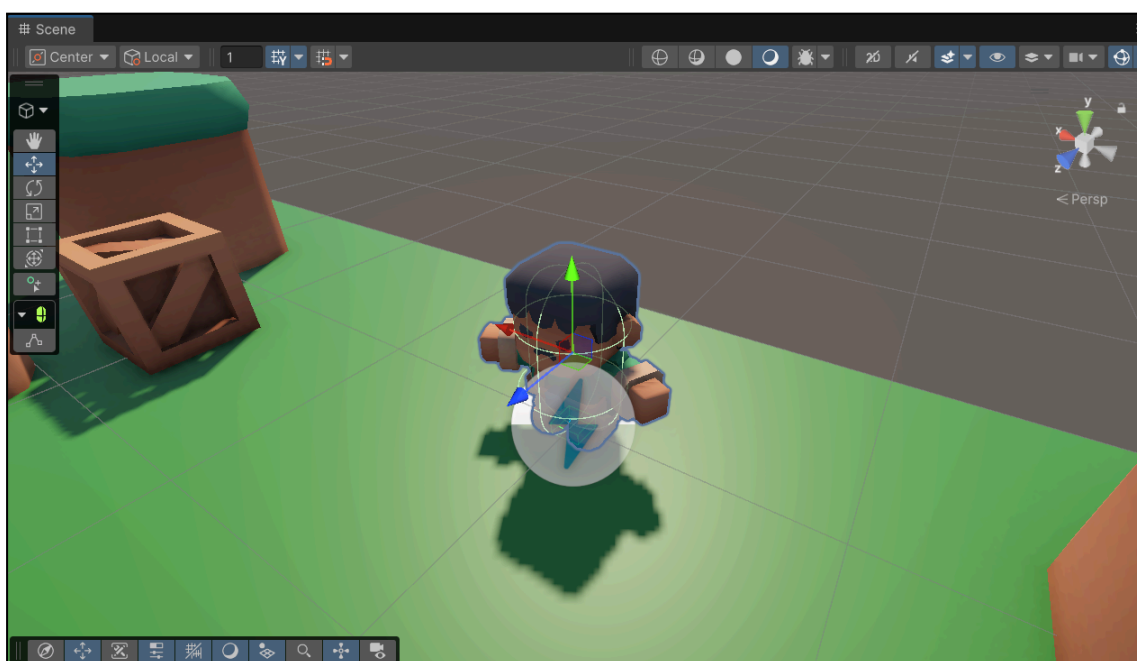
## 2. Creación del personaje y movimiento

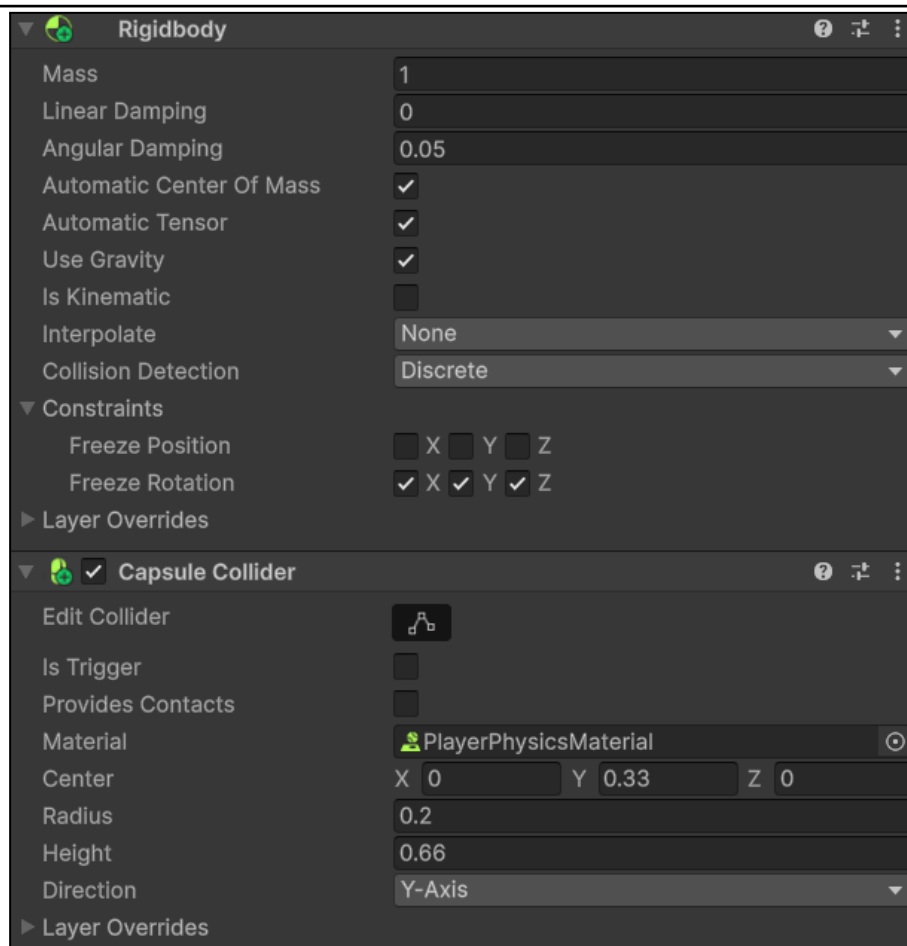
Se importaron unos modelos descargados de internet.

Fuente: <https://kenney.nl/assets/mini-characters>

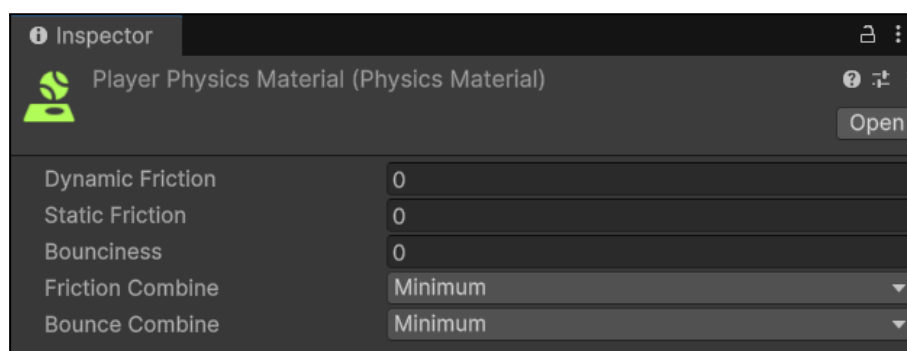


Se le agregó un **Rigidbody** para poder moverse utilizando físicas y un **Collider** para que interactúe con el entorno, en este caso es una cápsula, permitiendo un movimiento suave.

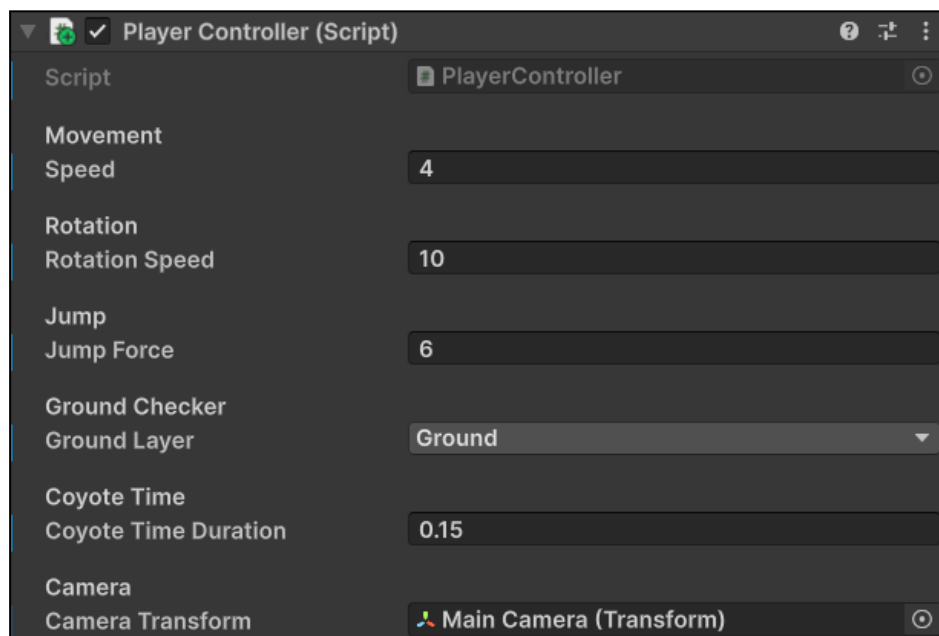




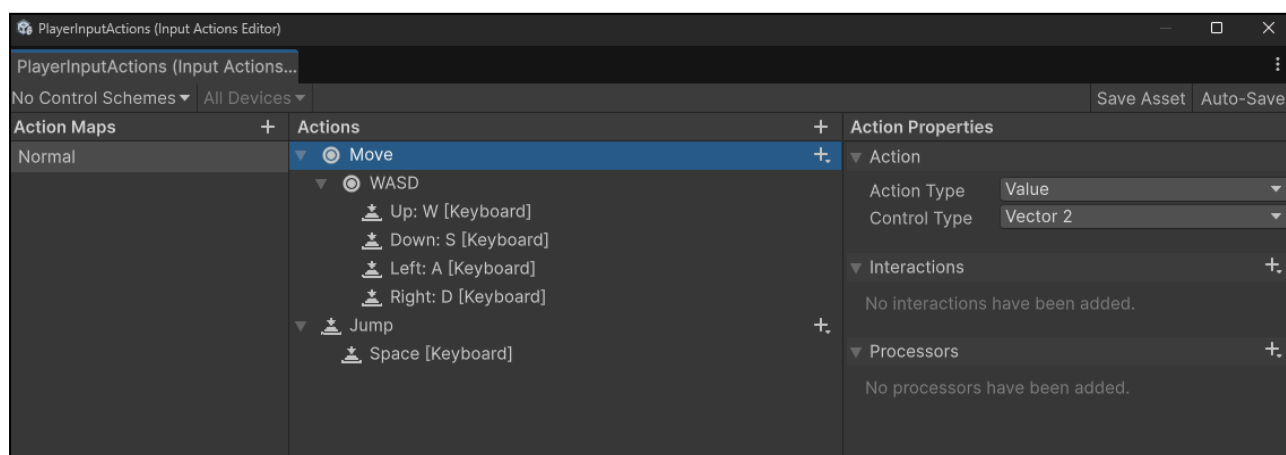
Como se puede ver en la imagen anterior, se le agregó un **Physics Material** a la cápsula de colisión y así evitar fricción con el entorno. Este **Physics Material** se configura aparte.



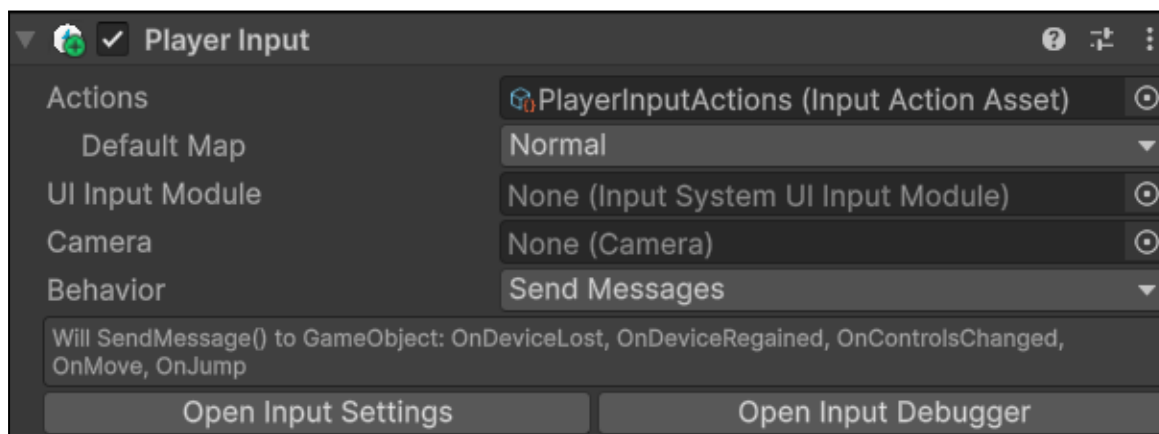
Para el movimiento se hace uso de los siguientes parámetros.



Se creó un archivo para las acciones de **Input** y así asignar los controles de movimiento y salto. Unity ya viene con un archivo similar por defecto, pero no se recomienda su uso, así que se creó uno personalizado con solamente lo necesario.



Se cuenta con el componente **Player Input** dentro del **Player** para gestionar las teclas presionadas y realizar acciones en correspondencia a estas.



El movimiento es relativo a la dirección en la cual está apuntando la cámara (luego se verá más a profundidad la configuración de la cámara con **Cinemachine**).

Primero tenemos las variables que controlan fuerzas y velocidades de las acciones del jugador, además de la referencia a la cámara. Estas son gestionadas desde el editor de Unity para mayor comodidad en general.

```

using UnityEngine;
using UnityEngine.InputSystem;

[RequireComponent(typeof(Rigidbody))]
public class PlayerController : MonoBehaviour {
    [Header("Movement")]
    [SerializeField] private float speed = 5f;

    [Header("Rotation")]
    [SerializeField] private float rotationSpeed = 10f;

    [Header("Jump")]
    [SerializeField] private float jumpForce = 5f;

    [Header("Ground Checker")]
    [SerializeField] private LayerMask groundLayer;

    [Header("Coyote Time")]
    [SerializeField] private float coyoteTimeDuration = 0.15f;

    [Header("Camera")]
    [SerializeField] public Transform cameraTransform;

```

|                                                                                    |                                                                                                                                                  |                                                                                     |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   | <p>UNIVERSIDAD NACIONAL DE SAN AGUSTIN<br/>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br/>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| <p>Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> |                                                                                                                                                  |                                                                                     |
| <p>Aprobación: 2022/03/01</p>                                                      | <p>Código: GUIA-PRLE-001</p>                                                                                                                     | <p>Página: 6</p>                                                                    |

Además, contamos con otras variables adicionales para el control del personaje, aquí destaca el **GroundCheckerSize** y el **GroundCheckerScale**, que sirven para crear un cubo que hará de detector de piso.

```
private const float GroundCheckerSize = 0.025f;
private readonly Vector3 GroundCheckerScale = new(2.5f, 1f, 2.5f);

private Rigidbody rb;
private Vector2 movementVector;
private float coyoteTimeCounter = 0f;
private bool jumpedThisFrame = false;
private bool onGround = true;
```

Tenemos el método inicial para detectar el **Rigidbody** y los métodos que se llaman cuando se presiona la tecla correspondiente según **Input Player**.

El **Coyote Time** permite saltar incluso si acabamos de separarnos del piso, dando margen de reacción adicional para mejorar la experiencia.

```
private void Awake() {
    rb = GetComponent<Rigidbody>();
    rb.freezeRotation = true;
}

public void OnMove(InputValue value) {
    movementVector = value.Get<Vector2>();
}

public void OnJump() {
    if (coyoteTimeCounter > 0f) {
        rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
        coyoteTimeCounter = 0f;
        jumpedThisFrame = true;
    }
}
```

|                                                                                    |                                                                                                                                                  |                                                                                     |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   | <p>UNIVERSIDAD NACIONAL DE SAN AGUSTIN<br/>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br/>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| <p>Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> |                                                                                                                                                  |                                                                                     |
| <p>Aprobación: 2022/03/01</p>                                                      | <p>Código: GUIA-PRLE-001</p>                                                                                                                     | <p>Página: 7</p>                                                                    |

El método núcleo es **FixedUpdate**, aquí se revisa si se está colisionando con el piso, se gestiona el **Coyote Time**, el movimiento y la rotación del personaje.

```
private void FixedUpdate() {
    CheckIfIsOnGround();
    HandleCoyoteTime();
    HandleMovement();
    HandleRotation();
}
```

La siguiente función se encarga de generar una caja de colisión para verificar si existe algo dentro de esta, en este caso se verifica la capa del piso ("**Ground**"), que está asignada a las plataformas del escenario creado.

```
private void CheckIfIsOnGround() {
    Vector3 center = transform.position;
    onGround = Physics.CheckBox(center, GroundCheckerScale * GroundCheckerSize,
    Quaternion.identity, groundLayer, QueryTriggerInteraction.Ignore);
}
```

La siguiente función controla el **Coyote Time**, evitando también casos de doble salto. Una vez que el jugador se separa del piso y empieza a caer de una plataforma, el tiempo de **Coyote Time** se va reduciendo hasta llegar a cero o menos, momento en el cual ya no podrá saltar hasta que toque el piso nuevamente y se reinicie dicho contador.

```
private void HandleCoyoteTime() {
    if (onGround && !jumpedThisFrame) {
        coyoteTimeCounter = coyoteTimeDuration;
    }
    else {
        coyoteTimeCounter -= Time.fixedDeltaTime;
    }
    jumpedThisFrame = false;
}
```

Las funciones **HandleMovement** y **HandleRotation** controlan el movimiento y la rotación del personaje como indica su nombre, y como se mencionó previamente, esto se hace relativo a la cámara, por lo que es importante contar con la referencia de esta.

Gracias a **GetCameraRelativeDirection** obtenemos la dirección en la que mira la cámara y tomamos esa de referencia. Se ignora el eje **Y** al que está mirando la cámara, porque no requerimos movernos en ese ángulo y sería contraproducente, ya que el **Player** intentaría atravesar el piso.

```
private void HandleMovement() {
    Vector3 direction = GetCameraRelativeDirection();
    Vector3 newLinearVelocity = direction * speed;
    newLinearVelocity.y = rb.linearVelocity.y;
    rb.linearVelocity = newLinearVelocity;
}
```

|                                                                                    |                                                                                                                                                  |                                                                                     |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   | <p>UNIVERSIDAD NACIONAL DE SAN AGUSTIN<br/>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br/>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| <p>Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> |                                                                                                                                                  |                                                                                     |
| <p>Aprobación: 2022/03/01</p>                                                      | <p>Código: GUIA-PRLE-001</p>                                                                                                                     | <p>Página: 8</p>                                                                    |

```
private void HandleRotation() {
    Vector3 direction = GetCameraRelativeDirection();
    if (direction.sqrMagnitude < 0.01f) return;

    Quaternion targetRotation = Quaternion.LookRotation(direction);
    rb.rotation = Quaternion.Slerp(rb.rotation, targetRotation, rotationSpeed *
Time.fixedDeltaTime);
}

private Vector3 GetCameraRelativeDirection() {
    Vector3 cameraForward = Vector3.ProjectOnPlane(cameraTransform.forward, Vector3.up).normalized;
    Vector3 cameraRight = Vector3.ProjectOnPlane(cameraTransform.right, Vector3.up).normalized;

    return cameraForward * movementVector.y + cameraRight * movementVector.x;
}
```

Finalmente contamos con una función de **debug** para dibujar el collider de cubo que se utiliza para detectar el piso. Esto es meramente visual.

```
private void OnDrawGizmosSelected() {
    Vector3 center = transform.position;
    Gizmos.color = onGround ? Color.green : Color.red;
    Gizmos.DrawWireCube(center, GroundCheckerSize * 2 * GroundCheckerScale);
}
```

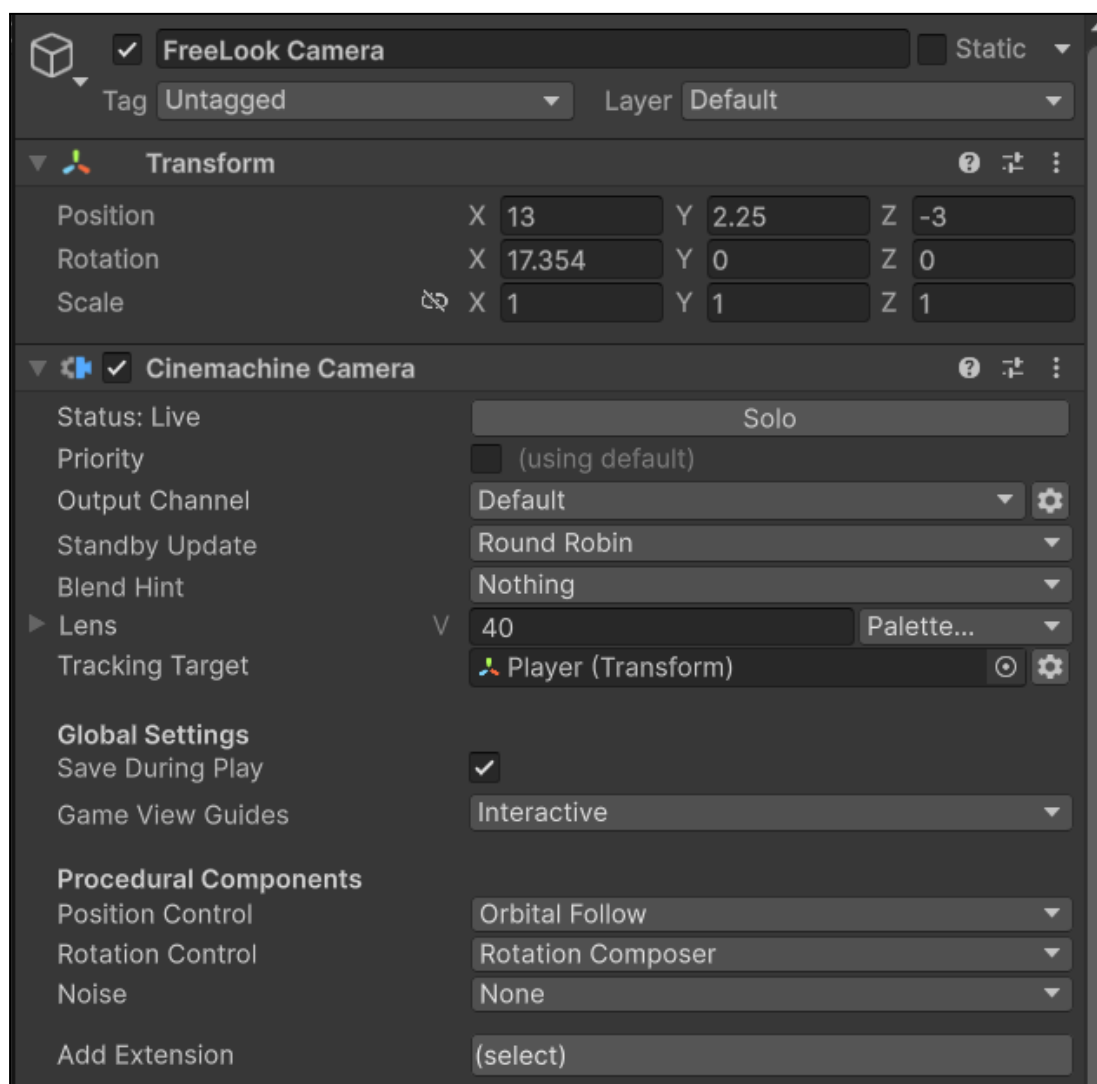
En el editor se ve así en los pies del jugador.



### 3. Cámara en Tercera Persona

Se hace uso de **Cinemachine**, plugin el cual nos permite agregar diferentes tipos de cámaras inteligentes según se requiera. En nuestro caso nos interesa la **Freelook Camera**. Solamente tenemos que asignar el objeto que debe seguir (el **Player** en este caso) y con las configuraciones por defecto basta para un resultado decente, pero se realizaron algunos cambios.

Por ejemplo, se modificó **Lens** para ver al personaje un poco más de cerca.



Además, se modificaron los ángulos límite de la cámara. En el eje **X** se puede rotar completamente, pero en el eje **Y** se limitó como se hace habitualmente en los juegos. También se aumentó el Offset en el eje **Y** para ver al personaje un poco más desde arriba.


☒ Cinemachine Orbital Follow

Binding Mode

World Space

Position Damping

X 1 Y 1 Z 1

Target Offset

X 0 Y 0 Z 0

Orbit Style

Three Ring

Top

Height 5 Radius 2

Center

Height 2.25 Radius 4

Bottom

Height 0.1 Radius 2.5

Spline Curvature

0.5

Recentring Target

Tracking Target (inactive)

Horizontal Axis

Value 0 Recenter

Center 0

Range -180 ... 180 ☒ Wrap

Recentring ☐

Vertical Axis

Value 17.5 Recenter

Center 17.5

Range -10 ... 45 ☐ Wrap

Recentring ☐

Radial Axis

1 Recenter


☒ Cinemachine Rotation Composer

Composition

Screen Position X 0 Y 0

Dead Zone ☐

Hard Limits ☐

Center On Activate ☒

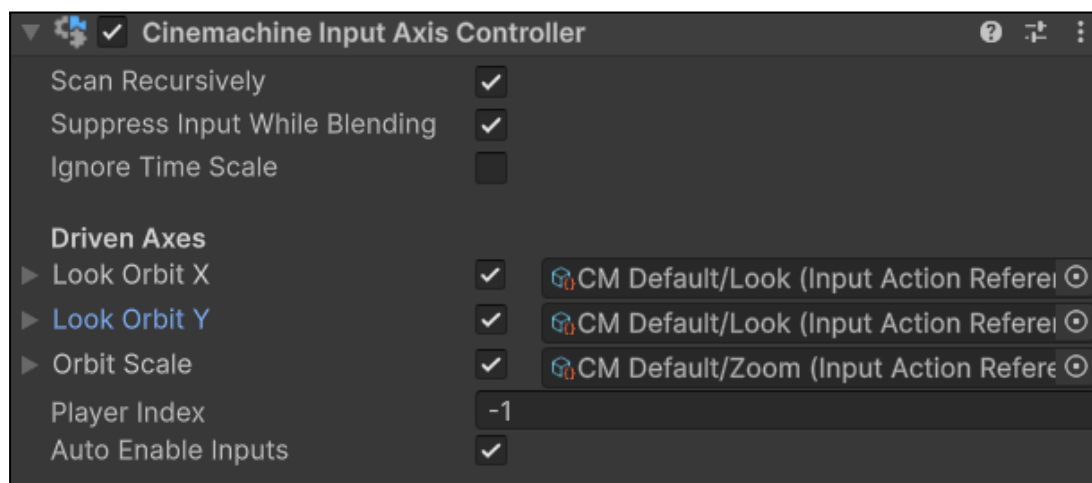
Target Tracking

Target Offset X 0 Y 1 Z 0

Damping X 0.5 Y 0.5

Lookahead ☐

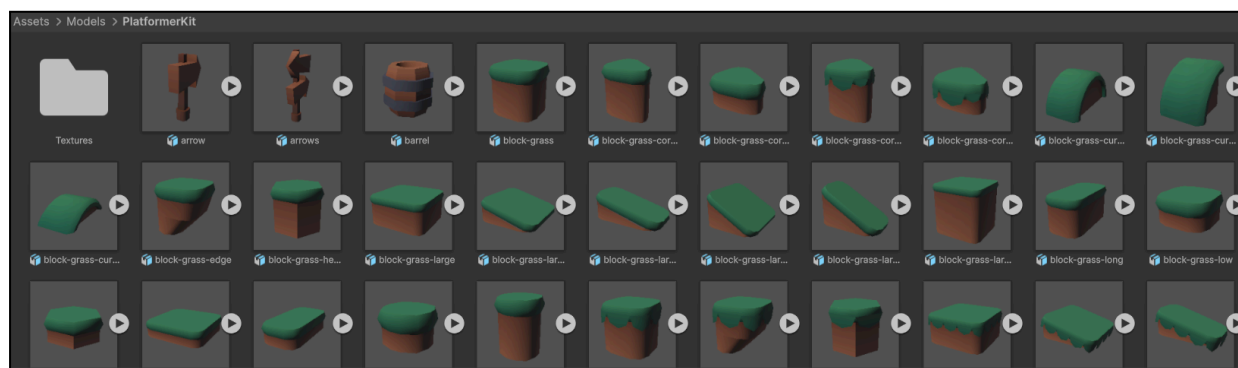
Finalmente, **Freelook Camera** cuenta con un controlador que funciona con el mouse, no se tiene que configurar nada más adicionalmente.



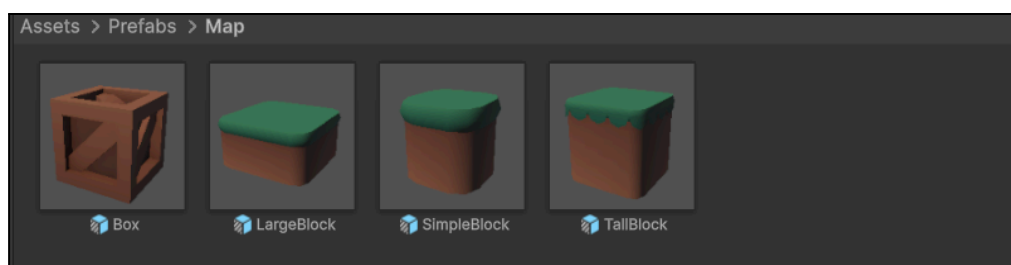
#### 4. Creación del escenario

Se descargaron unos assets para poder crear un escenario más agradable visualmente, en lugar de solamente usar los cubos por defecto.

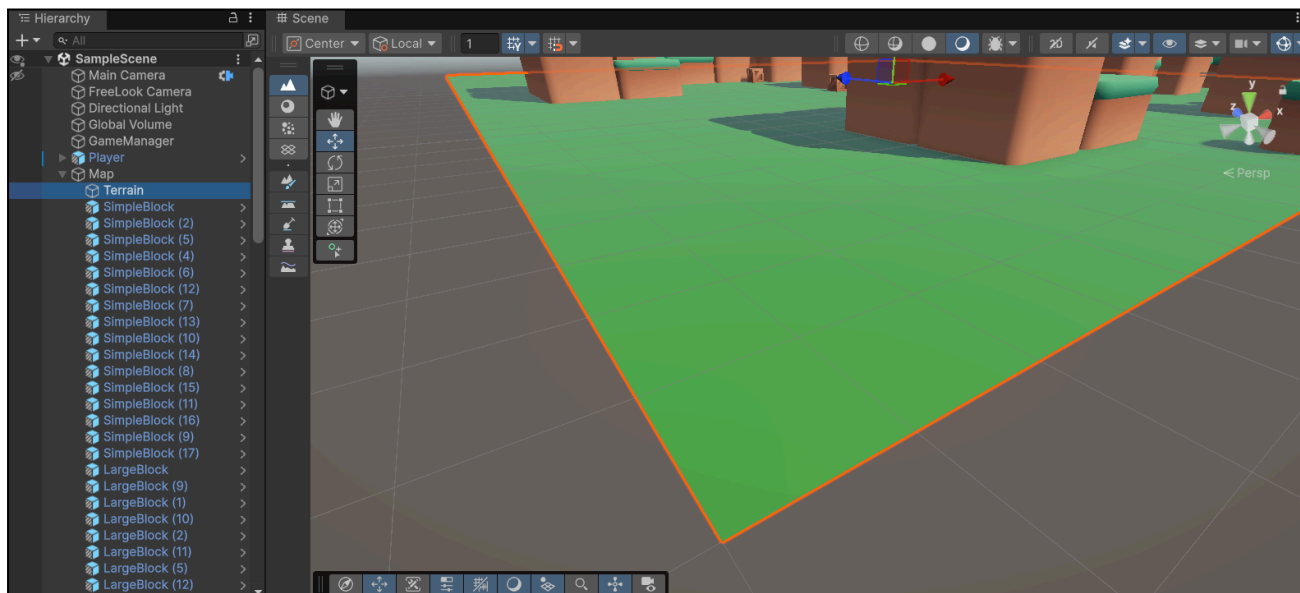
Fuente: <https://kenney.nl/assets/platformer-kit>



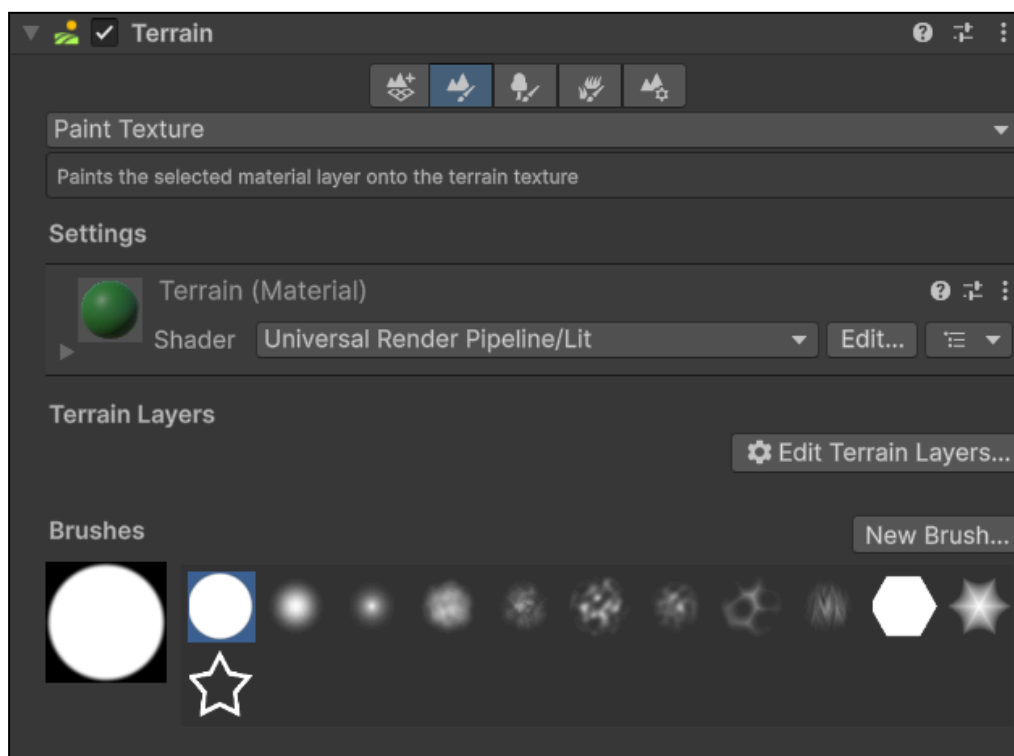
Se crearon prefabs con algunos de los bloques de este kit, agregándoles colisiones y las etiquetas respectivas, como **Ground** para que el jugador las detecte como piso.



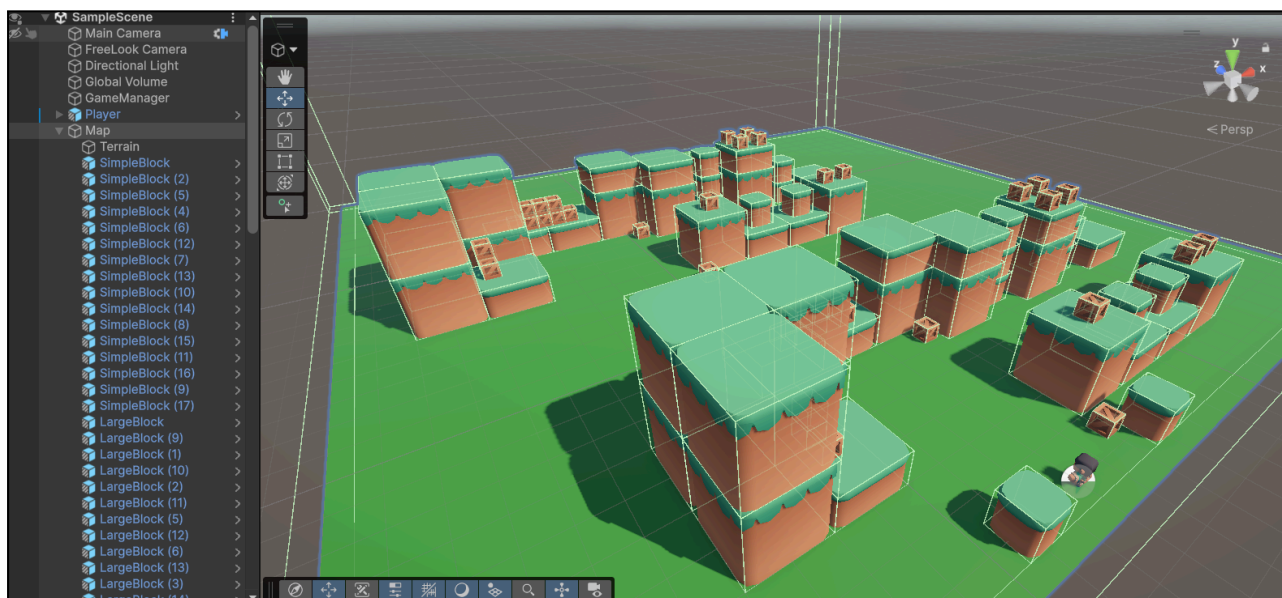
Se procedió a crear un escenario de prueba con los diferentes bloques. Todo se coloca sobre un **Terrain**, el cual actúa de un plano para el piso general.



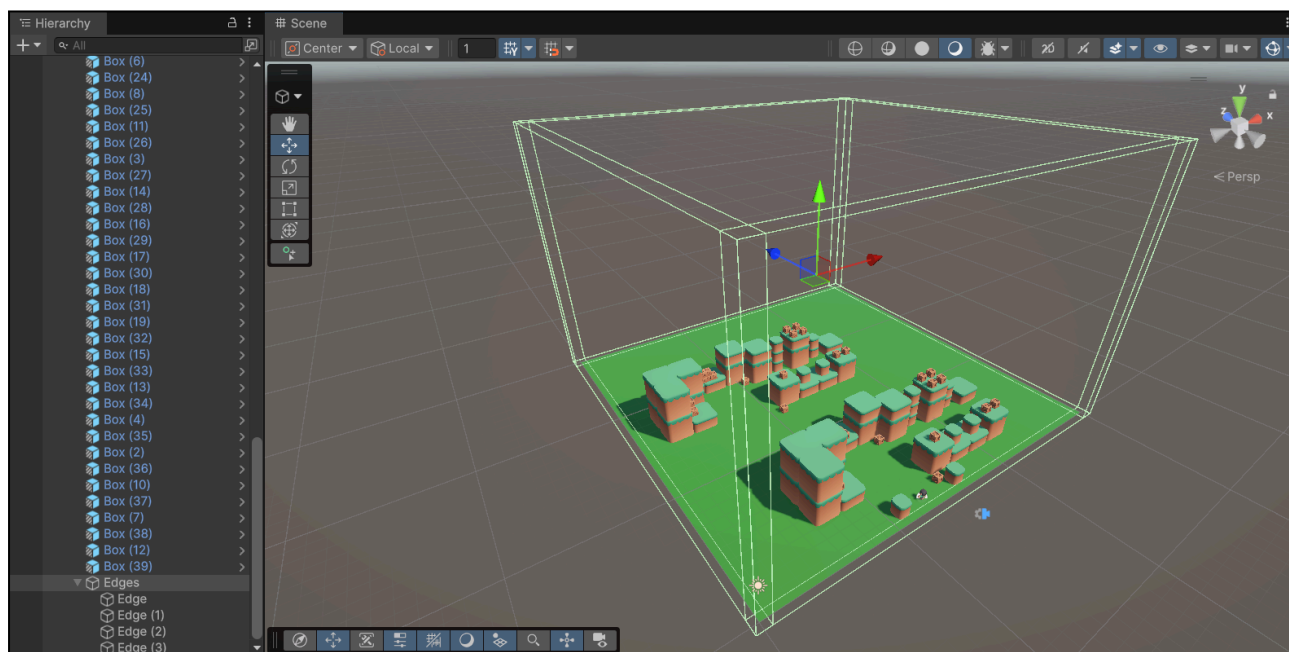
Su configuración es básica, se le asignó un material para darle color. Cuenta con bastantes parámetros, pero de momento nos viene bien como una base.



Se colocaron los bloques que tenemos como prefabs, sobre el terreno.



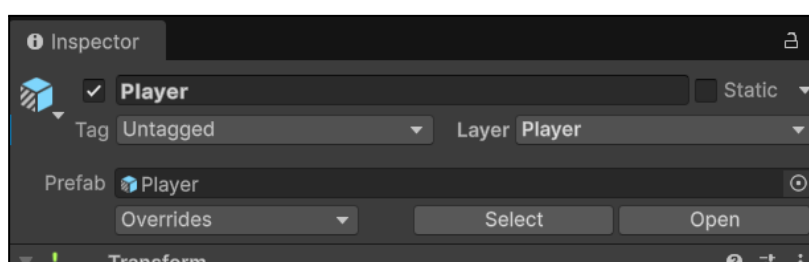
Adicionalmente, se agregaron unos colliders invisibles para delimitar el mapa.



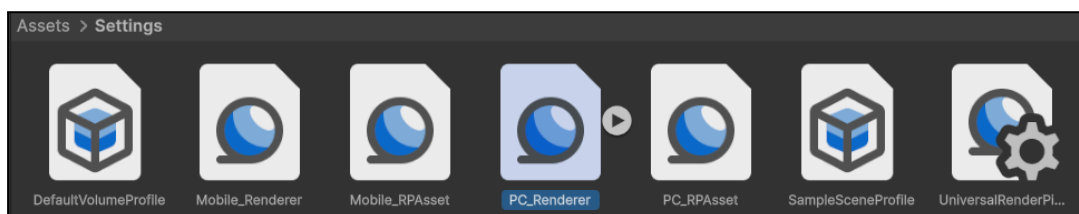
## 5. Efecto X-ray

Cuando la cámara sigue al jugador por el mapa, puede ocurrir que un bloque no permita ver al jugador, lo cual es típico. A veces se opta por dejarlo así, pero en este caso se implementó un efecto que permite ver al jugador a través de los bloques para mayor comodidad. Se aprovechó la capacidad de **Universal Render Pipeline (URP)** para conseguir esto.

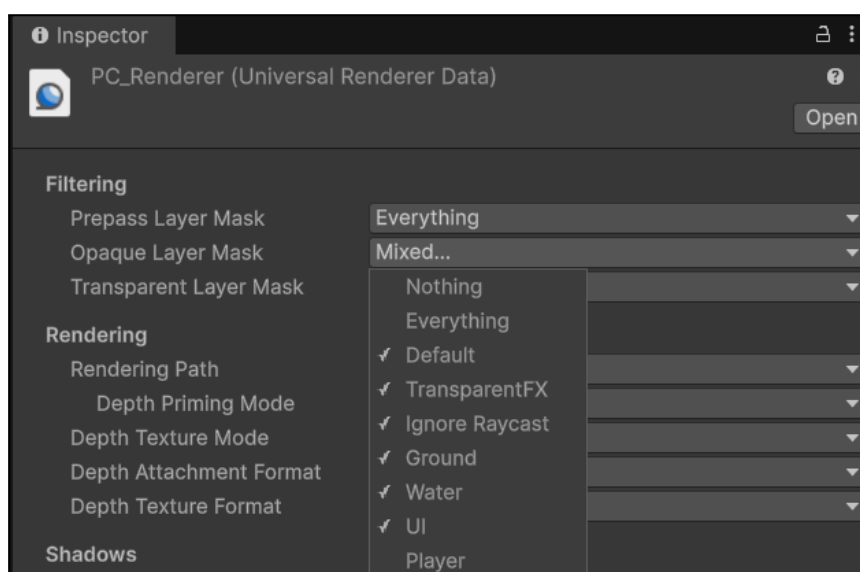
Primero le asignamos una capa al jugador, en este caso la capa (Layer) es **Player**.



Por consiguiente, nos dirigimos a la configuración de **URP**, estos archivos vienen por defecto al crear un proyecto en Unity con **URP**. Configuraremos únicamente para **PC** porque es el único sistema operativo objetivo ahora mismo. Para otros dispositivos, se debe modificar el archivo correspondiente.



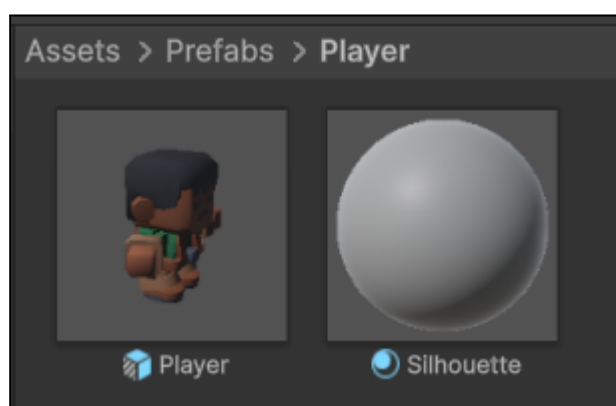
Removemos la capa **Player** de **Opaque Layer Mask** para que ya no renderice dicha capa.



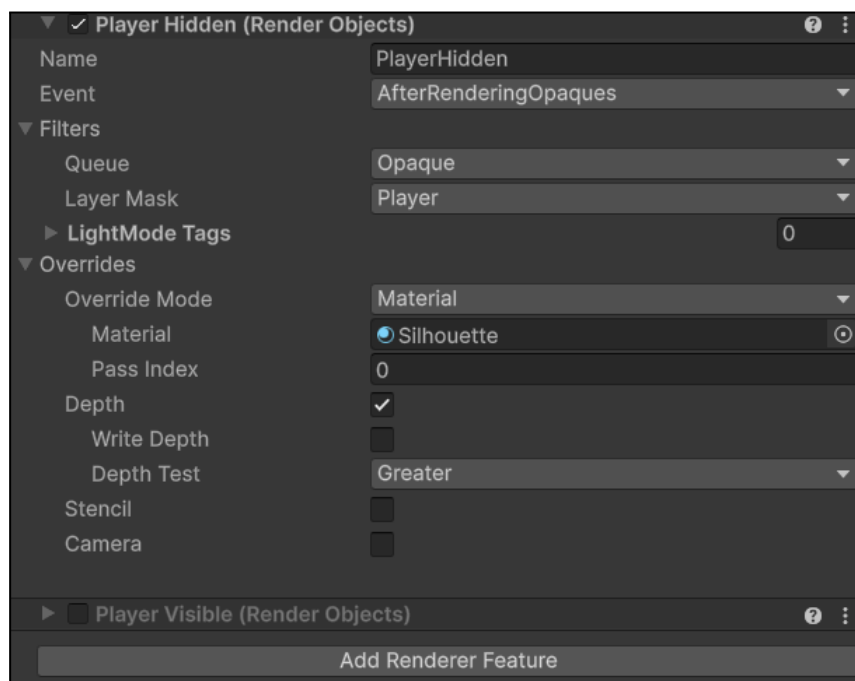
Solamente se ve la sombra.



Creamos un material que hará de silueta del jugador.



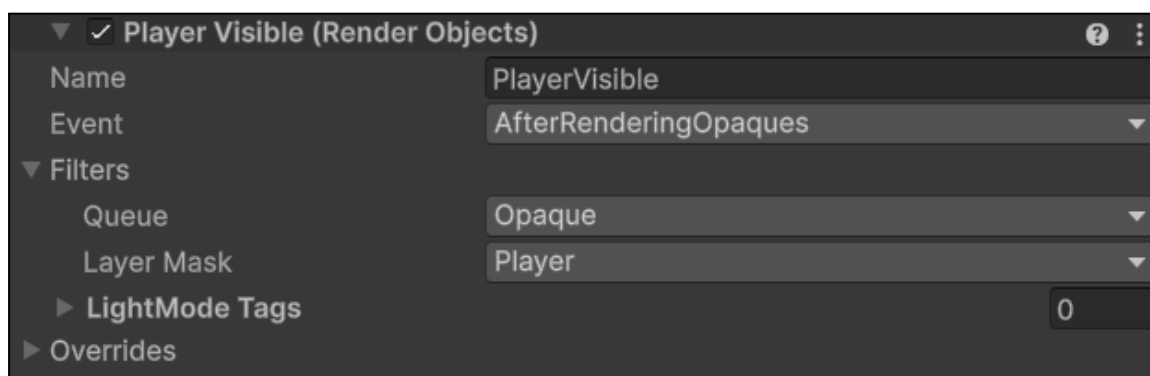
Agregamos ahora una “pasada” para el jugador en modo oculto, es decir que le indicaremos qué dibujar cuando el jugador esté detrás de alguna de las otras capas. Se le asigna el material creado.



Ahora mismo solo se ve la silueta a través de los objetos, pero no se ven los colores originales.



Para solventar lo anterior, le damos una segunda “pasada”, esta es para dibujar al jugador de la manera habitual, además de la silueta, esa ya está dibujada con la primera “pasada”. Esta simplemente va con la configuración por defecto.



Ahora se ve como se tenía planeado originalmente.



## 6. Enlace al video de demostración

<https://drive.google.com/file/d/163cwvmjQ5xCJZ-nFH0pRNGgjuP5X5DIk/view?usp=sharing>

|                                                                                           |                                                                                                                                                  |                                                                                     |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|          | <p>UNIVERSIDAD NACIONAL DE SAN AGUSTIN<br/>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br/>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| <p><b>Formato:</b> Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> |                                                                                                                                                  |                                                                                     |
| <p><b>Aprobación:</b> 2022/03/01</p>                                                      | <p><b>Código:</b> GUIA-PRLE-001</p>                                                                                                              | <p><b>Página:</b> 18</p>                                                            |